The Journal of Open Source Software

OGX: An Open-Source, Vendor-Neutral Generative AI Application Server

Francisco Javier Arceo [1¶], **Sébastien Han**[1], **Matthew Farrellee**[3], **Charlie Doern**[1], **Yuan Tang**[1], **Derek Higgins**[1], **Varsha Prasad Narsing** [1], **Gordon Sim**[1], **Sumanth Kamenani**[1], **Ben Browning**[1], **and Raghotham Murthy**[2]

1 Red Hat AI, USA **2** Meta, USA **3** Independent ¶ Corresponding author

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 05 June 2026
Published: unpublished

Summary

OGX (Open GenAI Stack), formerly Llama Stack (Llama Stack Contributors, 2025), is an open-source AI application server and Python library that implements the APIs of major frontier labs (OpenAI, Anthropic, Google) with pluggable backend providers (OGX Contributors, 2026b). Teams building agentic AI applications—such as retrieval-augmented generation (RAG) pipelines, multi-turn conversational agents, and tool-calling workflows—can develop against a single, stable API surface and deploy with any combination of inference engine, vector database, and safety backend, without changing application code.

OGX's primary API focus is the Responses API for server-side agentic orchestration, conforming to the Open Responses specification (Open Responses Community, 2026). The server also exposes Chat Completions, Embeddings, Vector Stores, Files, and Batches endpoints. Beyond OpenAI compatibility, OGX natively supports the Anthropic Messages API (`/v1/messages`) and Google GenAI Interactions API (`/v1alpha/interactions`), allowing teams using any of the three major client SDKs to connect to the same server. OGX supports over 20 inference providers (including vLLM, Ollama, OpenAI, Anthropic, Bedrock, and Gemini), 13 vector store backends, and 7 safety providers. It can run as an HTTP server for production deployments or be imported directly as a Python library for scripting and notebooks. A companion Kubernetes Operator (OGX Contributors, 2026a) automates deployment lifecycle management through custom resources, supporting multi-architecture builds, hot-swappable distribution images, and both shared and per-tenant isolation topologies. Together, OGX and its operator serve as the self-hosted, model-agnostic backend for AI-powered developer tools such as Claude Code, Codex CLI, OpenCode, and OpenHands.

Statement of Need

AI application development today is tightly coupled to proprietary API providers. While inference-only workloads can increasingly be swapped across providers—vLLM, for example, supports the Responses API for basic inference—applications that rely on the full stack (retrieval, tool calling, conversation state, safety guardrails) remain difficult to migrate without rewriting significant application logic. This coupling limits reproducibility, makes comparisons across model providers difficult, and prevents teams from running AI workloads on controlled infrastructure—a requirement in regulated, privacy-sensitive, and air-gapped environments.

Existing open-source tools address parts of this problem but not the whole. Inference engines like vLLM (Kwon et al., 2023) and SGLang (Zheng et al., 2024) serve models efficiently but do not provide retrieval, tool calling, or conversation management. Gateway proxies like LiteLLM route requests across providers but do not execute the agentic loop or manage vector stores. Client-side frameworks like LangChain (LangChain, Inc., 2023) and LangGraph (LangChain,

Inc., 2024) provide developer abstractions but push orchestration, state management, and security enforcement to the application layer.

OGX fills this gap by providing a complete, self-hosted AI application server that implements the OpenAI API surface—with the Responses API as its primary focus—alongside Anthropic Messages and Google GenAI Interactions compatibility layers. Developers write code against standard endpoints (`/v1/responses`, `/v1/chat/completions`, `/v1/vector_stores`) and swap the underlying infrastructure through configuration, not code changes. This decouples three decisions that are currently entangled: which SDK to use, which model to run, and where to deploy.

State of the Field

The AI application ecosystem has stratified into layers that each solve a subset of the deployment problem. OGX continues the Llama Stack project under a renamed, model-agnostic mission; the relevant comparison is therefore the server-side API layer that OGX provides versus adjacent inference engines, gateways, and client-side frameworks.

Inference engines (vLLM (Kwon et al., 2023), SGLang (Zheng et al., 2024), Ollama) focus on efficient model serving. They optimize throughput and latency but do not provide retrieval, conversation state, tool execution, or safety guardrails. An application using vLLM for inference must separately integrate a vector database, implement its own agentic loop, and manage multi-turn state.

API gateways (LiteLLM, OpenRouter) provide a unified interface across multiple inference providers but act as pass-through proxies. They do not manage vector stores, execute tool calls, or maintain conversation history—they translate request formats between SDKs and providers.

Client-side frameworks (LangChain (LangChain, Inc., 2023), LangGraph (LangChain, Inc., 2024), LlamaIndex (LlamaIndex, 2022), CrewAI (CrewAI, Inc., 2024), Haystack (deepset, 2023)) provide rich developer abstractions for building agents and RAG pipelines. However, they execute orchestration client-side, distributing security-critical logic across application code. These frameworks are complementary to OGX: they compose agent logic while OGX provides the server-side execution target they call into.

Proprietary platforms (OpenAI's Responses API (OpenAI, 2025), Databricks Mosaic AI (Databricks, 2025)) offer integrated experiences but couple applications to a specific vendor's infrastructure and pricing.

OGX occupies a distinct position: a self-hosted, vendor-neutral server that implements the full API surface—inference, retrieval, tool execution, conversation management, and safety—with pluggable providers at every layer. Its conformance to the Open Responses specification (Open Responses Community, 2026) ensures interoperability with any client that speaks the same protocol. It does not compete with client-side frameworks; it is the infrastructure they deploy against when reproducibility, provider portability, and centralized policy enforcement matter.

Software Design

Provider Architecture

OGX's core abstraction is the pluggable provider. Each API capability (inference, vector storage, safety, tool runtime, file processing) is defined by a Protocol interface in the lightweight `ogx-api` package, allowing third-party providers to implement the contract without depending on the full server. Concrete providers implement these interfaces for specific backends: `remote::openai` and `remote::anthropic` for hosted APIs, `remote::vllm` for self-hosted GPU inference, `inline::faiss` and `remote::pgvector` for vector search, and so on. A routing layer

dispatches requests to provider instances based on logical resource identifiers, enabling multiple providers to serve the same API simultaneously—for example, Ollama handling local models while OpenAI handles hosted models, both behind `/v1/chat/completions`.

A *distribution* packages a specific set of providers and configuration into a deployable unit, decoupling application logic from infrastructure selection. This design favors reproducible configuration over ad hoc application code: teams can prototype with lightweight inline providers (Ollama, sqlite-vec (Garcia, 2024)) and deploy to production backends (vLLM, pgvector) by changing the distribution, not the application.

Dual Deployment Model

OGX runs in two modes. **Server mode** exposes HTTP endpoints accessible from any language or tool. **Library mode** allows direct Python import with zero network overhead, suitable for notebooks and scripts. Both modes use identical provider routing and API semantics.

Multi-SDK Compatibility

OGX serves three client SDK protocols from a single server. The **OpenAI-compatible endpoints** (`/v1/chat/completions`, `/v1/responses`, `/v1/vector_stores`) are the primary interface and the Responses API implementation conforms to the Open Responses specification (Open Responses Community, 2026). The **Anthropic Messages endpoint** (`/v1/messages`) and **Google GenAI Interactions endpoint** (`/v1alpha/interactions`) provide native compatibility for teams using those SDKs. This portability comes with a trade-off: OGX must normalize provider-specific behavior into stable API contracts while still exposing enough backend-specific configuration for real deployments.

Server-Side Agentic Orchestration

The Responses API implements server-side agentic orchestration: the inference-tool-inference loop executes within the server process, not the client (OpenAI, 2025). This centralizes security enforcement, tool authorization, and conversation state management, at the cost of moving some flexibility from application code into server configuration. Built-in tools include file search (RAG over vector stores), web search, code interpretation, and Model Context Protocol (MCP) (Anthropic, 2026) integration for external tool servers.

OGX is extending server-side execution with a Containers API and a Skills API. The Containers API (`/v1/containers`) manages sandboxed execution environments—matching the OpenAI Containers API—enabling models to execute shell commands in isolated Docker, Podman, or Kubernetes containers via a `shell` tool in the Responses API. The Skills API (`/v1alpha/skills`) manages versioned skill bundles that package tools, prompts, and configuration into reusable, composable units. Together, these APIs move code execution and task composition onto the server, extending the same trust-boundary principle that governs retrieval and tool authorization.

Unlike inference engines and API gateways that treat requests as stateless, OGX manages state natively: the Conversations API persists multi-turn history with tenant-scoped isolation, the Prompts API provides versioned prompt templates, and a resource registry tracks models, vector stores, and files as first-class server objects. A Compaction API summarizes long histories to manage context window limits. Telemetry is built on OpenTelemetry (OTEL) with MLflow (Zaharia et al., 2018) tracing integration for logging spans, tool calls, and retrieval steps to existing ML experiment tracking infrastructure. This state and observability layer is what makes OGX a complete application server rather than a stateless proxy.

For multitenant deployments, OGX provides attribute-based access control (ABAC) that enforces tenant isolation at the retrieval, tool execution, state management, and API routing layers. The security properties of this architecture have been formally analyzed and empirically validated (Arceo & Narsing, 2026).

Kubernetes Operator

The OGX Kubernetes Operator (OGX Contributors, 2026a) provides declarative deployment through the `OGXServer` custom resource. A single CR specifies the distribution, replica count, storage, inference backend, and network policies. The operator manages full lifecycle reconciliation with support for both vanilla Kubernetes and OpenShift, ConfigMap-driven image overrides for fleet-wide updates, multi-architecture builds (amd64/arm64) with FIPS-compliant images, and shared instances with ABAC isolation, per-tenant namespace isolation, and hybrid topologies.

Example Usage

The following example demonstrates building a RAG agent using the standard OpenAI SDK against an OGX server. The same code works regardless of which inference provider or vector store backend is configured:

```python
from openai import OpenAI

client = OpenAI(base_url="http://localhost:8321/v1", api_key="unused")

# Create a vector store and upload documents
vector_store = client.vector_stores.create(name="docs")
with open("manual.pdf", "rb") as file:
    client.vector_stores.files.upload(vector_store_id=vector_store.id, file=file)

# Query with server-side RAG via the Responses API
response = client.responses.create(
    model="meta-llama/Llama-3.2-3B-Instruct",
    input="What are the installation requirements?",
    tools=[{"type": "file_search", "vector_store_ids": [vector_store.id]}],
)
print(response.output_text)
```

Switching from a local Ollama backend to a production vLLM cluster requires changing only the server's distribution configuration—the client code above remains identical. Published tutorials demonstrate this portability across diverse enterprise backends, including IBM watsonx.ai with Milvus vector storage (IBM Community, 2025) and Oracle Cloud Infrastructure with OCI AI Blueprints (Oracle, 2025).

Research Impact Statement

OGX has realized impact through both public deployments and customer production use. Publicly documented examples under the former Llama Stack name include an intelligent OpenShift operations agent combining RAG, web search, and MCP tool integration for automated incident response (Red Hat, 2025), enterprise RAG pipelines on IBM watsonx.data with Milvus (IBM Community, 2025), and generative AI application development on Oracle Cloud Infrastructure (Oracle, 2025). Maintainers report production deployments with customers in telecommunications, semiconductor manufacturing, financial services, insurance, and consulting. The framework has been presented at Meta Connect (Meta, 2024) and IBM TechXchange (Clyburn, 2025) as a standardization layer for enterprise AI applications.

The security architecture—specifically, the multitenant isolation model combining ABAC-gated retrieval, server-side orchestration, and pluggable provider backends—was formally analyzed in a peer-reviewed publication at the ACM Conference on AI and Agentic Systems (Arceo & Narsing, 2026). OGX conforms to the Open Responses specification (Open Responses

Community, 2026) and serves as a reference implementation for open, vendor-neutral agentic AI APIs.

As of June 2026, the project has over 8,400 GitHub stars, 242 contributors, 4,000 commits, and 68 releases across nearly two years of public development. Community engagement includes weekly contributor calls, an active Discord server, and integrations contributed by external organizations including Red Hat, IBM, Oracle, and Infinispan.

AI Usage Disclosure

Generative AI tools, including GitHub Copilot and Anthropic Claude models available during development, were used for code completion, documentation drafting, and paper drafting. Assistance was limited to generating candidate text or code that human contributors reviewed, edited, tested, and validated. Core architectural decisions, API design, the security model, and final paper content were made by human authors.

Acknowledgements

We thank Meta for creating and open-sourcing Llama Stack, now renamed OGX. We are grateful to Red Hat for supporting the development of OGX through employee time and infrastructure support. We thank the OGX contributor community for their sustained contributions to the project.

References

Anthropic. (2026). *Model context protocol specification*. https://modelcontextprotocol.io/specification/

Arceo, F. J., & Narsing, V. P. (2026). Securing the agent: Vendor-neutral, multitenant enterprise retrieval and tool use. *Proceedings of the ACM Conference on AI and Agentic Systems*, 862–872. https://doi.org/10.1145/3786335.3813145

Clyburn, C. (2025). *Llama stack: Kubernetes for RAG and AI agents in generative AI*. https://mediacenter.ibm.com/media/Llama+Stack+Kubernetes+for+RAG+AI+Agents+in+Generative+AI/1_xl78upq2

CrewAI, Inc. (2024). *CrewAI: Framework for orchestrating role-playing autonomous AI agents*. https://github.com/crewAIInc/crewAI

Databricks. (2025). *Mosaic AI Agent Framework*. https://www.databricks.com/product/machine-learning/retrieval-augmented-generation

deepset. (2023). *Haystack: End-to-end LLM framework for building production-ready applications*. https://github.com/deepset-ai/haystack

Garcia, A. (2024). *Sqlite-vec: A vector search SQLite extension*. https://github.com/asg017/sqlite-vec

IBM Community. (2025). *Build RAG with Llama Stack and watsonx.data Milvus*. https://community.ibm.com/community/user/blogs/divya13/2025/05/08/build-rag-with-llama-stack-and-watsonxdata-milvus

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., & Stoica, I. (2023). Efficient memory management for large language model serving with PagedAttention. *Proceedings of the 29th ACM Symposium on Operating Systems Principles*, 611–626. https://doi.org/10.1145/3600006.3613165

LangChain, Inc. (2023). *LangChain: Build context-aware reasoning applications*. https://github.com/langchain-ai/langchain

LangChain, Inc. (2024). *LangGraph: Build resilient language agents as graphs*. https://github.com/langchain-ai/langgraph

Llama Stack Contributors. (2025). *Llama stack*. https://github.com/llamastack/llama-stack

LlamaIndex. (2022). *LlamaIndex: Data framework for LLM applications*. https://github.com/run-llama/llama_index

Meta. (2024). *Llama stack: Chapter one*. https://developers.facebook.com/m/meta-connect-developer-sessions/llama-stack-chapter-one/

OGX Contributors. (2026a). *OGX kubernetes operator*. https://github.com/ogx-ai/ogx-k8s-operator

OGX Contributors. (2026b). *OGX (open GenAI stack)*. https://github.com/ogx-ai/ogx

Open Responses Community. (2026). *Open responses specification*. https://www.openresponses.org/

OpenAI. (2025). *Responses API reference*. https://platform.openai.com/docs/api-reference/responses

Oracle. (2025). *Accelerating enterprise gen AI applications development on OCI with Llama Stack and OCI AI blueprints*. https://blogs.oracle.com/ai-and-datascience/accelerating-enterprise-gen-ai-applications-development-on-oci-with-llama-stack-and-oci-ai-blueprints

Red Hat. (2025). *Generative AI applications with Llama Stack: A notebook-guided journey to an intelligent operations agent*. https://www.redhat.com/en/blog/generative-ai-applications-llama-stack-notebook-guided-journey-intelligent-operations-agent

Zaharia, M. A., Chen, A., Davidson, A., Ghodsi, A., Hong, S. A., Konwinski, A., Murching, S., Nykodym, T., Ogilvie, P., Parkhe, M., Xie, F., & Zumar, C. (2018). Accelerating the Machine Learning Lifecycle with MLflow. *IEEE Data Eng. Bull.*, *41*, 39–45. https://api.semanticscholar.org/CorpusID:83459546

Zheng, L., Yin, L., Xie, Z., Huang, J., Sun, C., Yu, C. H., Cao, S., Kober, C., Shi, L., Wu, C.-S., Zhang, H., Sheng, Y., Gonzalez, J. E., Stoica, I., & Ma, W.-L. (2024). SGLang: Efficient execution of structured language model programs. *Advances in Neural Information Processing Systems*, *37*.